

JCST Papers

Only for Academic and Non-Commercial Use

Thanks for Reading!



[Survey](#)

[Computer Architecture and Systems](#)

[Artificial Intelligence and Pattern Recognition](#)

[Computer Graphics and Multimedia](#)

[Data Management and Data Mining](#)

[Software Systems](#)

[Computer Networks and Distributed Computing](#)

[Theory and Algorithms](#)

[Emerging Areas](#)



JCST WeChat

Subscription Account

JCST URL: <https://jcast.ict.ac.cn>

SPRINGER URL: <https://www.springer.com/journal/11390>

E-mail: jcast@ict.ac.cn

Online Submission: <https://mc03.manuscriptcentral.com/jcast>

Twitter: JCST_Journal

LinkedIn: Journal of Computer Science and Technology

A Survey of Quantization in LLM: Unlocking Potential Hardware Efficiency

Yi-Dong Chen¹ (陈逸东), Kai-Jun Zheng¹ (郑楷颢), Zhen-Hua Guo² (郭振华), Qi-Hao Zhang¹ (张齐颢)
Yong-Hua Zhang¹ (张永华), and Ji-Dong Zhai¹ (翟季冬), *Distinguished Member, CCF, Senior Member, IEEE*

¹ Department of Computer Science and Technology, Tsinghua University, Beijing 100190, China

² Inspur Electronic Information Industry Co., Ltd., Beijing 100190, China

E-mail: chenqidong@tsinghua.edu.cn; kaijun823@tsinghua.edu.cn; guozhenhua@ieisystem.com; zqh23@mails.tsinghua.edu.cn
zhangyonghua@tsinghua.edu.cn; zhajidong@tsinghua.edu.cn

Received September 24, 2025; accepted December 29, 2025.

Abstract Large language models (LLMs) have achieved remarkable progress in natural language processing, but their immense scale leads to significant computational and storage overheads, limiting their deployment and widespread application in resource-constrained environments. Model quantization, as an effective model compression technique, significantly reduces LLMs' memory footprint and computational requirements by lowering the numerical precision of model parameters and/or activations, while striving to maintain minimal performance loss. This survey aims to comprehensively review the latest advancements in LLM quantization, covering various techniques from the pretraining phase to the inference phase. We will delve into state of the art quantization during pretraining, post-training quantization, and quantization-aware training in quantization fine-tuning, and various quantization methods during inference. Through in-depth analysis of these methods, this survey seeks to provide researchers and engineers with a comprehensive understanding of LLM quantization techniques to identify future research directions and offers an insight of how to generate high performance low-precision kernels in different chips.

Keywords quantization, large language model (LLM), mixed-precision, kernel generation

1 Introduction

In recent years, large language models (LLMs), such as the GPT series, LLaMA^[1, 2], and BLOOM^[3], have demonstrated unprecedented performance in various natural language processing tasks such as question answering, text summarization, and machine translation, due to their exceptional language understanding and generation capabilities. However, the parameter count of these models typically ranges from billions to hundreds of billions, leading to immense computational resources and memory requirements during training and inference. For example, the

DeepSeek-V3 671B model^[4] requires approximately 1 400 GB of memory in floating-point 16 (FP16) precision, which exceeds the capacity of most single GPUs, posing severe challenges for the deployment and application of LLMs^[1]. As shown in Fig.1, over the past eight years, the scale of models has grown rapidly, but the growth of GPU memory size has been relatively slow.

To address these challenges, model compression techniques have emerged, among which model quantization is considered one of the most effective and widely applied methods. Quantization significantly reduces model memory footprint, accelerates inference

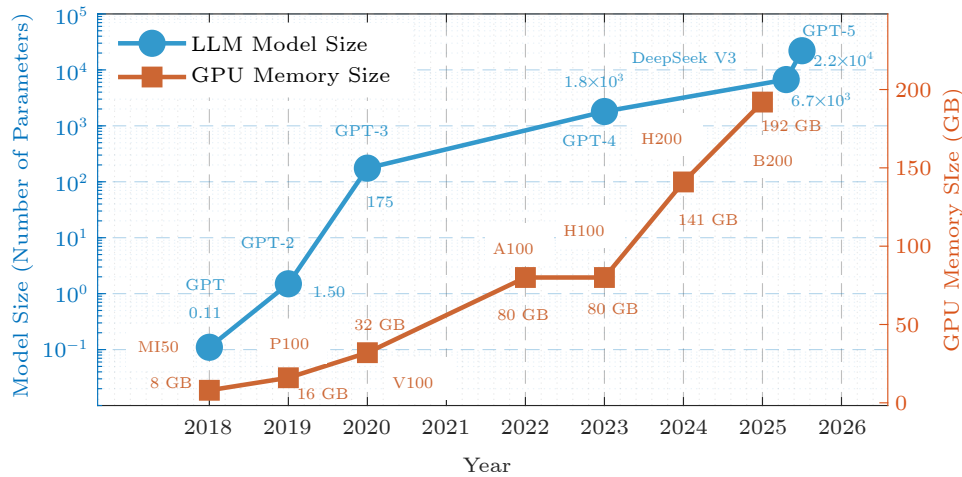


Fig.1. LLM model size vs GPU memory development (2018–2025), showing fast increase in model scale and slow increase in memory size.

speed, and reduces energy consumption by converting model parameters (e.g., weights) or intermediate computation results (e.g., activations) from high precision (e.g., floating-point 32 (FP32) or FP16) to low-precision representations (e.g., 8-bit integers INT8, 4-bit integers INT4^[5–8], or even lower-bit floating-point formats like floating-point 8 (FP8), floating-point 4 (FP4)). This enables LLMs to run on edge devices, consumer-grade hardware, or in cloud environments sensitive to latency and cost^[9, 10].

The development of LLM quantization techniques spans multiple stages of the model lifecycle, including pretraining, fine-tuning, and inference. In the pretraining phase, researchers explore using low-precision data types (e.g., FP8, INT8, FP4, and INT4) directly for model training, aiming to reduce computational costs and memory requirements from the outset. In the fine-tuning phase, post-training quantization (PTQ)^[5, 11, 12] and quantization-aware training (QAT)^[7] are two primary strategies that introduce quantization after model training or during the training process to optimize the performance of quantized models. In the inference phase, various quantization methods are proposed to optimize different components (e.g., weights, activations, key-value (KV) cache), aiming to maximize compression and acceleration while maintaining performance. As shown in Fig.2, an increasing number of chips support low-precision computing units, which gives developers the opportunity to utilize hardware more efficiently.

2 Background

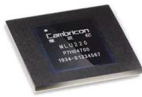
LLMs have learned and mastered complex lan-

guage patterns, world knowledge, and reasoning abilities, demonstrating exceptional performance in various tasks such as text generation, question answering, machine translation, and code generation, sometimes even reaching human-level capabilities. For example, the DeepSeek-V3 671B model, with its 671 billion parameters, exhibits remarkable few-shot learning abilities. However, this improvement in capability is accompanied by a dramatic increase in the scale of the model, with parameter counts ranging from billions to hundreds of billions, which poses significant challenges for the practical deployment and application of LLMs^[13].

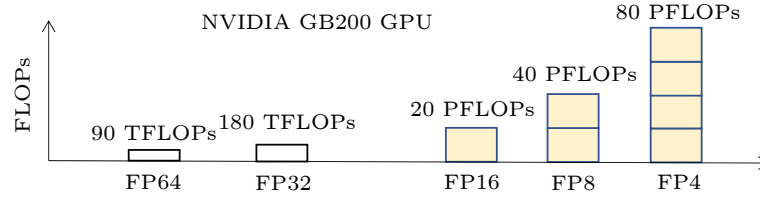
To overcome these challenges, researchers have actively explored various model compression techniques, among which model quantization is considered one of the most effective and promising methods^[14, 15]. Fig.3 presents a taxonomy of LLM quantization, categorizing techniques across three key stages: pretraining, fine-tuning, and inference.

In the pretraining stage, aspects such as computation, communication, and various weight types (master weight, gradient weight, backward weight) are considered, along with optimizers. Optimization methods like Automatic Mixed Precision (AMP)^[16], Transformer Engine^[17], Microsoft Automatic Mixed Precision (MS-AMP)^[16], and DeepSeek^[4] are also involved, aiming to enhance the efficiency of quantization during the pretraining process.

For the fine-tuning stage, it encompasses two main paradigms: PTQ (with methods like BiLLM^[11] and QEFT^[18]) and QAT (featuring techniques such as Torchao^[19] and QATLoRA^[7]). These approaches strive to adapt pretrained models to quantization

Chips					
	Ascend 910	MLU220	DCU Z100	NVIDIA 4090	NVIDIA GB200
Precision	FP16	FP16	FP16	FP16	FP16
	INT8	INT8	INT8	INT8	INT8
			INT4	INT4	—
				FP8	FP8
					FP4

(a)



(b)

Fig.2. Low-precision computing unit in different chips. (a) Different chips. (b) Peak performance of GB200 GPU.

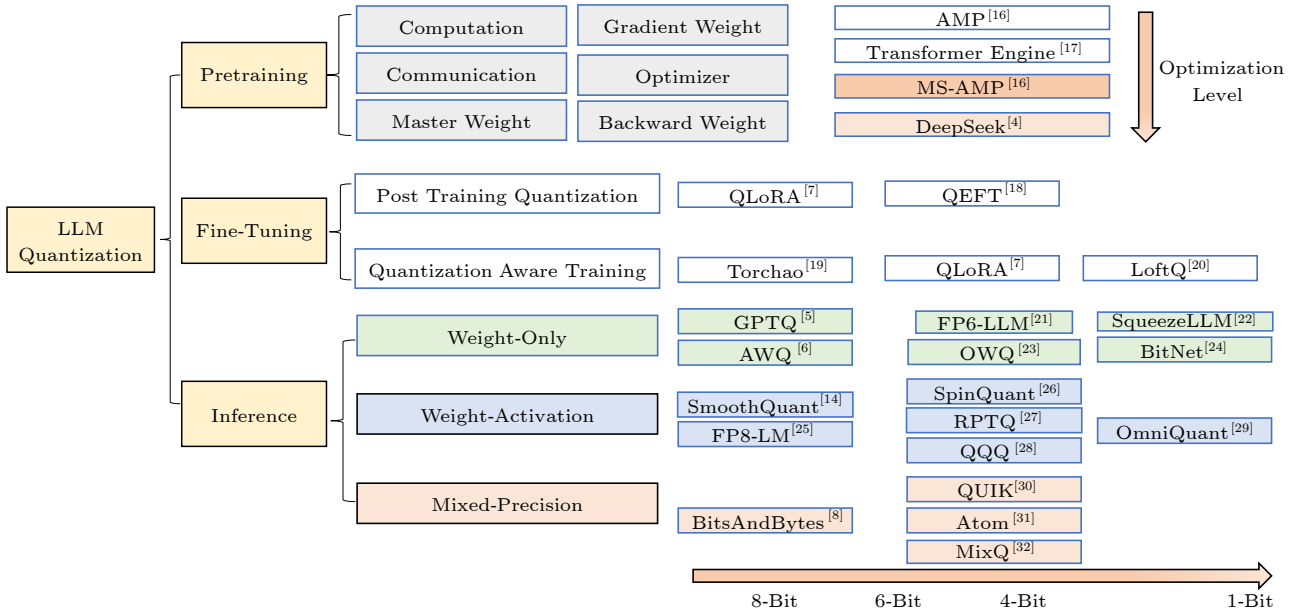


Fig.3. Overview of LLM quantization.

while preserving performance.

In the inference stage, quantization techniques are further divided into three categories: 1) weight-only (e.g., GPTQ^[5], AWQ^[6]), 2) weight-activation (e.g., SmoothQuant^[14], FP8^[25]), and 3) mixed-precision (e.g., BitsAndBytes^[8], MixQ^[32]). Additionally, a precision spectrum from 8-bit down to 1-bit is illustrated, reflecting the trade-off between model precision and computational efficiency in inference scenarios.

This survey aims to provide a comprehensive overview and in-depth analysis of LLM quantization methods. We will systematically introduce and compare various mainstream quantization techniques based on the stage of quantization and the objects being quantized. Specifically, this survey will cover the following aspects.

1) Quantization in pretraining: discussing the application of low-precision formats like FP8, INT8,

INT4, and FP4 in LLM pretraining, along with their challenges and innovations;

2) Quantization in fine-tuning: analyzing the principles, advantages, and limitations of post-training quantization (PTQ) and quantization-aware training (QAT)^[33–35];

3) Quantization in inference: quantization strategies for different model components, including weight-only (AWQ, GPTQ), weight and activation (SmoothQuant, RPTQ^[27]), activation quantization with outliers^[36], and KV cache quantization.

Through an in-depth exploration of these key techniques, this survey aims to provide readers with a holistic perspective on LLM quantization, helping to understand current research hotspots, technical challenges, and code generation of mixed-precision kernel^[37, 38]. Finally, this survey will summarize the current challenges facing LLM quantization and outline future research directions.

3 Basic Details of Quantization

3.1 Mathematical Details of Quantization

All quantization schemes rely on a core mathematical framework for mapping high-precision values (x_{fp}) to low-precision quantized values (x_q) and reverse-dequantizing back to an approximate high-precision value ($\hat{x}_{fp}$) for training/inference. The core operations include scaling functions, rounding rules, error characteristics, and quantization granularity; these form the foundational math of quantization and apply universally to both weight and activation quantization.

3.1.1 Scaling Factor of Quantization

Quantization transforms high precision values to low-precision values. This is the most fundamental formulation, with two variants (symmetric and asymmetric) for different data distributions.

Symmetric Quantization. Assuming data is centered around 0, ideal for weight tensors with symmetric value ranges, the quantization step is formulated as $x_q = \text{round}(x_{fp}/s)$, and the dequantization step is formulated as $\hat{x}_{fp} = x_q \times s$, where $s = \max(|x_{fp}|)/(2^{b-1} - 1)$ is the scaling factor, b denotes the bitwidth of the low-precision format (e.g., $b = 4$ for INT4, $b = 8$ for INT8). The denominator $2^{b-1} - 1$ defines the maximum integer value for signed INT.

Asymmetric Quantization. Accounts for non-zero-centered data by introducing a zero-point (z_p) to align the zero with a quantized integer value, the quantization step is formulated as

$$x_q = \text{round}\left(\frac{x_{fp} - \min(x_{fp})}{s}\right) + z_p,$$

and the dequantization step is formulated as $\hat{x}_{fp} = (x_q - z_p) \times s$, where $s = (\max(x_{x_{fp}}) - \min(x_{fp})) / (2^b - 1)$, $z_p \in \mathbb{Z}$ is the zero-point^[39].

3.1.2 Error Characteristics

Quantization error is defined as $\epsilon = x_{fp} - \hat{x}_{fp}$, a composite of clipping error (ϵ_c) and rounding error (ϵ_r): $\epsilon = \epsilon_c + \epsilon_r$. The mathematical properties of error propagation govern quantization's impact on model accuracy, with two key characteristics.

Static Error. Weights are quantized once (offline); therefore, error is fixed and additive ($W_q = W_{fp} + \epsilon_W$). For linear layers ($y = Wx + b$), the error propagates linearly: $\hat{y} = (W_{fp} + \epsilon_W)x + b = y + \epsilon_W x$. Error magnitude scales with activation values x .

Dynamic Error. Activations are quantized per inference batch (runtime); thus, error is stochastic ($A_q = A_{fp} + \epsilon_A$). For deep networks, error propagates cumulatively across layers (error stacking), amplified by non-linear activation functions (ReLU, GELU). Low-bit quantization (INT4) exacerbates this, requiring error compensation (e.g., quantization-aware training, QAT).

3.1.3 Quantization Granularity

Quantization granularity refers to the scope of tensor elements that share a single scaling factor (s) and zero-point (z_p). It is a critical design choice: finer granularity reduces quantization error but increases computation overhead, while coarser granularity maximizes acceleration, but may degrade accuracy. All granularity schemes are built on the core affine scaling formulation above, differing only in how s/z_p are assigned to tensor subsets. Granularity is categorized by tensor dimension and model component, including per-layer, per-channel, per-group quantization.

Per-Layer Quantization. Per-layer quantization has the coarsest practical granularity; it offers minimal overhead but results in the highest error, and is thus used only for lightweight models with uniform tensor distributions.

Per-Channel Quantization. It is the most widely adopted granularity for weights. A single s (and op-

tional z_p) is assigned to each channel of a weight tensor.

Per-Group Quantization. It is a middle ground between per-channel and per-layer. Tensor elements are partitioned into small groups (e.g., 128 elements per group), with one s/z_p per group. Per-group quantization is widely used for INT4 weight quantization in LLMs, balancing error and overhead^[40].

3.2 Quantization Schemes

This subsection explores several prominent quantization strategies, each with distinct trade-offs in accuracy, compression ratio, and hardware compatibility. We begin with weight-only quantization (WOQ), which offers a simpler approach with minimal accuracy impact, then discuss WOQ for more comprehensive acceleration, and finally address the critical challenge of activation quantization with outliers, where advanced techniques like mixed-precision quantization are employed to maintain high accuracy.

3.3 Weight-Only Quantization

WOQ is an efficient model compression technique. Its core idea is to quantize the weights in a neural network from high precision (e.g., FP32/FP16) to low precision (e.g., INT8/INT4/INT2)^[5] while keeping the activations at their original precision. This method offers significant advantages during the inference phase. First, it drastically reduces model storage space and memory bandwidth usage. Second, it significantly accelerates computations on hardware that supports integer arithmetic (such as CPUs and dedicated NPUs (neural processing units)).

Fig.4 shows the roofline of weight-only quantization. In this figure, the vertical axis represents the operational intensity (Ops/s), and the horizontal axis denotes the ratio of operations to elements ($Ops/Elements$). In the memory bound regime, as the ratio of operations to elements increases, the operational intensity rises until it reaches the peak operations level of FP16. This indicates that with weight-only quantization, the memory bottleneck is alleviated to some extent, allowing the system to approach the peak computational capability more quickly compared with full FP16 computation in this regime since the dequantization step finished in the register file (RF) (Fig.5).

Since activations are not quantized, weight-only quantization generally ensures high model accuracy while avoiding the complex calibration and accuracy

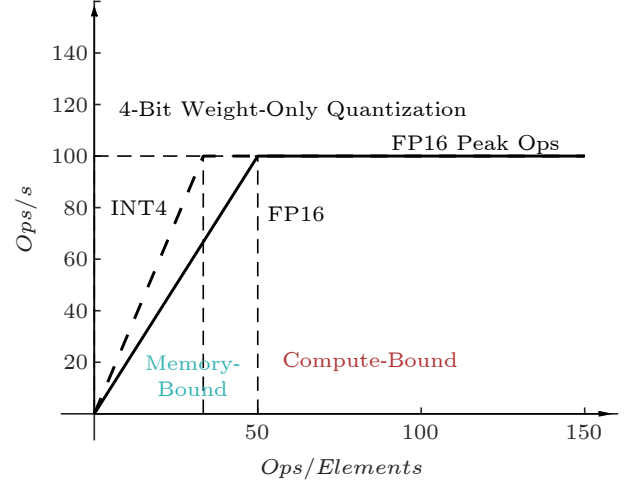


Fig.4. Roofline of weight-only quantization.

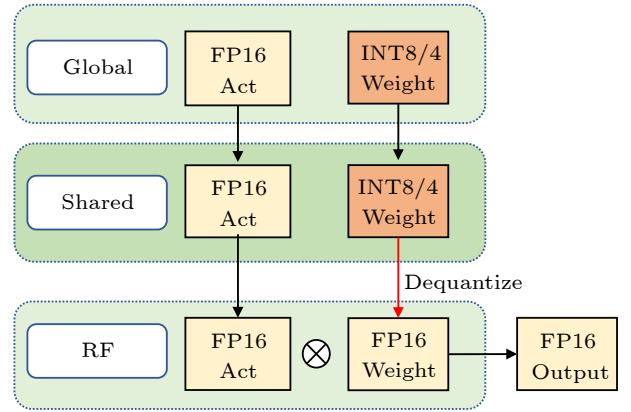


Fig.5. Benefit of weight-only quantization. By quantizing only the weights, this approach reduces model storage requirements and memory bandwidth usage.

loss issues associated with activation quantization. In recent years, research has yielded fruitful results in areas including quantization strategies, quantization granularity, post-training quantization (PTQ), quantization-aware training (QAT), and hardware co-design.

3.3.1 Weight-Activation Quantization

Weight-Activation (WxAx) quantization refers to quantizing both the model's weights and activations^[41–44]. This method can achieve comprehensive model compression and inference acceleration because it can reduce model storage precision, memory bandwidth, and computational operations. However, simultaneously quantizing weights and activations is generally more challenging than weight-only quantization because activations have a larger dynamic range and often contain outliers.

Fig.6 shows the roofline of weight-activation quantization. In the memory bound region, as the ratio of

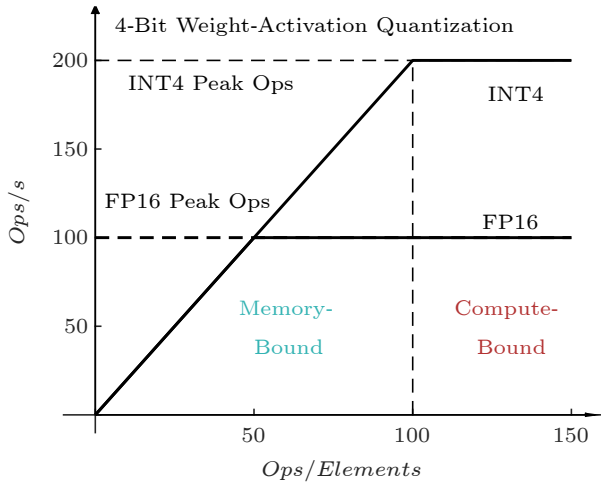


Fig.6. Roofline of weight-activation quantization.

operations to elements increases, the operational intensity rises and reaches a peak operational level corresponding to INT4 precision. When transitioning to the compute-bound region, we could use the low precision computing units as shown in Fig.7. The 4-bit weight-activation quantization maintains its peak operations level, which is 4 times higher than that of the peak operations level of FP16. This indicates that in the compute bound scenario, weight-activation quantization can leverage the reduced computational complexity from lower-precision representations to achieve a greater computational throughput compared with full precision (FP16) computation.

However, the weight-activation quantization scheme must be co-designed with hardware characteristics and quantization algorithm itself. Specialized quantization schemes for future architectures are important (e.g., Mamba, MoE). For example, logarithmic quantization^[39] can better match weight distribution in certain scenarios, but it still cannot be widely used on general-purpose hardware. The design of low-precision computing unit may break through the limi-

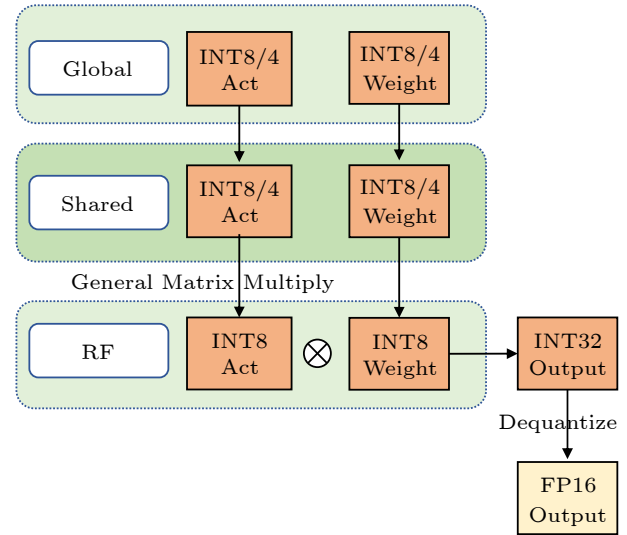


Fig.7. Benefit of weight-activation quantization. Both the weight and activation are quantized. The low-precision computing unit could be used.

tations of current uniform quantization.

3.3.2 Activation Quantization with Outliers

Outliers are prevalent in the activations of LLMs, where a few activation values are much larger than others^[45–49]. These outliers greatly affect quantization accuracy because they stretch the quantization range, causing most non-outlier values to be assigned to very few quantization levels, resulting in information loss.

Mixed-precision quantization^[36] has emerged as a sophisticated solution to this challenge. The workflow is shown in Fig.8. Unlike uniform quantization schemes that apply the same bit-width across all layers or tensors, mixed-precision quantization assigns different bit-widths to different layers or tensors based on their sensitivity to quantization errors. This approach recognizes that not all parts of a neural network contribute equally to model accuracy, and

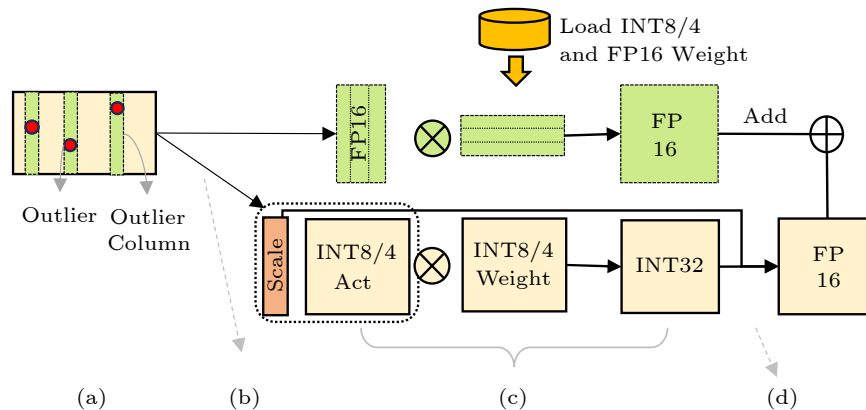


Fig.8. Workflow of mixed-precision computing. (a) Outlier identification. (b) Quantize. (c) Compute. (d) Dequantize.

strategically allocates higher precision to more sensitive layers while aggressively quantizing less critical components^[50–54].

As shown in Table 1, the performance of mixed-precision schemes (Mix(w4a16o16) and Mix(w4a8-o16)) on diverse tasks (such as Pip for fill-in-blank, Winogrande for commonsense reasoning, Humaneval for programming, Gsm8k for math, and Lambda for logic) is close to that of FP16, which is a full precision representation. In contrast, weight-only quantization methods like AWQ(w4a16) and GPTQ(w4a16) exhibit significant accuracy drops, especially on complex tasks like Humaneval, Gsm8k, and Lambda. This indicates that mixed-precision quantization can strike a better balance between reducing the precision (and thus computational and memory costs) and maintaining model accuracy, making it a more favorable choice when accuracy preservation is crucial along with efficiency gains. The code is publicly available on Github^①, and we also show how to implement custom quantization methods in our project. All the quantized and FP16 models could be downloaded directly in the HuggingFace^{②③④}.

4 Pretraining with Quantization

pretraining the LLM requires immense resources: hundreds of GPUs, weeks of computation, and vast amounts of memory not only for model weights but also for optimizer states, gradients, and activations.

FP16 and brain floating-point 16 (BF16) precision have become standards to reduce memory footprint and increase computational throughput compared with FP32.

Quantization during pretraining, as opposed to PTQ, aims to train a model from scratch in lower precision. This is a more challenging endeavor due to the need to maintain numerical stability throughout the optimization process. The key is to strategically apply quantization to different components of the training stack, as illustrated in Table 2. The choice of precision for each component (computation, communication, master weights, gradient, etc.) varies across methods.

4.1 Quantization Schemes in Pretraining

Modern deep learning pretraining, especially for LLMs, employs sophisticated mixed-precision and quantization schemes to balance performance and efficiency. These schemes often involve a careful selection of precision for different components of the model and training process. Below, we detail several prominent approaches, analyzing their strategies for quantizing computation, communication, weights, master weights, weight gradients, and optimizer states.

4.1.1 FP16 Automatic Mixed Precision

FP16 automatic mixed precision (AMP) is a wide-

Table 1. Evaluating the Accuracy (%) of Different Quantization Methods Across Different Datasets

Method	Pip (Fill in Blank)	Winogrande (Fill in Blank)	Humaneval (Programming Problem)	Gsm8k (Math)	Lambda (Logic Problem)
FP16	74.10	61.40	50.61	64.74	60.50
Mix(w4a16o16) ^[32]	73.79	59.10	50.22	63.24	59.43
Mix(w4a8o16) ^[8]	73.29	58.10	51.22	62.24	60.43
AWQ(w4a16) ^[6]	73.35	57.01	26.22	55.57	44.56
GPTQ(w4a16) ^[5]	74.53	57.34	31.35	53.55	45.21

Table 2. Pretraining Quantization with Different Precision

Method	Computation	Communication	Weight	Master Weight	Gradient	Optimizer
FP16 AMP	FP16	FP32	FP32	FP32	FP32	FP32
Transformer Engine	FP8	FP32	FP32	FP32	FP32	FP32
MS-AMP O1 ^[16]	FP8	FP8	FP16	FP32	FP8	FP32
MS-AMP O2	FP8	FP8	FP16	FP32	FP8	FP8/FP16
MS-AMP O3	FP8	FP8	FP8	FP16	FP8	FP8/FP16
Deepseek ^[4]	Block FP8	FP8	FP8	FP32	FP8	BF16

^①https://github.com/Qcompiler/llm_bench, Jan. 2026.

^②<https://www.modelscope.cn/models/deepseek-ai/DeepSeek-V2-Lite-Chat>, Jan. 2026.

^③<https://www.modelscope.cn/models/TechxGenus-MS/DeepSeek-V2-Lite-Chat-AWQ>, Jan. 2026.

^④<https://huggingface.co/ModelCloud/DeepSeek-V2-Lite-gptq-4bit>, Jan. 2026.

ly adopted technique that combines FP16 and FP32 numerical formats to accelerate deep neural network training while preserving accuracy. It leverages the benefits of half-precision (reduced memory, increased bandwidth, faster math on tensor cores) by performing most operations in FP16, while critical steps are maintained in FP32 to ensure numerical stability.

4.1.2 FP8-FP16 Mixed Precision

Transformer Engine (TE)^[17] introduces FP8 computation for transformers. It dynamically scales tensors to maximize the use of the FP8 range, preventing overflow and underflow. However, it remains conservative by keeping weights, master weights, gradients, and optimizer states in higher precision (FP32). Its primary benefit is a compute speedup without altering the core training dynamics. TE^[17] utilizes two distinct FP8 datatypes: E4M3 (1 sign bit, 4 exponent bits, 3 mantissa bits) for higher precision, typically used for forward activations and weights, and E5M2 (1 sign bit, 5 exponent bits, 2 mantissa bits) for higher dynamic range, often used for gradients in the backward pass.

4.1.3 Pretraining with FP8

Microsoft (MS-AMP)^[16] proposed progressive Automatic mixed precision optimization levels (O1, O2, O3) to enhance deep learning training efficiency by integrating 8-bit collective communication, optimizer state quantization, and distributed parallel training. FP8-LM^[55] also explores training LLMs with FP8 precision, demonstrating the feasibility of FP8 training for LLMs.

Optimization Level O1. It introduces FP8 for both computation and communication, tackling two major bottlenecks. Weights are stored in FP16, while master weights and optimizer states remain in FP32 for stability. Gradients are quantized to FP8, reducing memory during the backward pass.

Optimization Level O2. It builds on O1 by further quantizing the optimizer states to FP8/FP16. This is a major step for memory reduction, as optimizer states often consume the most memory. The

master weight remains in FP32 to safeguard the update process.

Optimization Level O3. It is the most aggressive level, quantizing weights to FP8 and master weights to FP16. This removes the need for a full FP32 master copy, yielding the highest memory savings. Stability is maintained through careful scaling and management of the FP16 master weights.

DeepSeek^[4] reported training a 671B parameter model using a custom quantization strategy. The DeepSeek team uses block FP8, a variant of FP8 that uses a separate scaling factor for blocks of values within a tensor, offering higher accuracy. Notably, they use FP8 for computation, communication, weights, and gradients. They keep the master weight in FP32 for stability but use BF16 for the optimizer states. BF16, with its larger dynamic range compared with FP16, is a prudent choice for optimizer moments, helping to avoid overflow during update calculations.

Both COAT^[56] and MOSS^[57] address the fundamental challenge that while FP8 precision offers a theoretical 2x speedup and memory reduction, its limited numerical range and precision enable model accuracy and training stability. They move beyond simple quantization by introducing novel scaling methods to manage these trade-offs effectively.

4.2 Trade-Offs of Different Precision in Pretraining Steps

The evolution from FP16 AMP to MS-AMP O_3 and DeepSeek's method represents a fundamental trade-off: memory and communication efficiency versus numerical stability, as listed in Table 3. Retention of a high-precision Master Weight (FP32 in most cases) is a common thread across all but the most aggressive methods, acting as an anchor for stability. Quantizing the optimizer state is arguably the highest impact change for memory reduction but introduces the risk of corrupting the optimization trajectory. Techniques like dynamic scaling (as in Transformer Engine) and blockwise quantization (as in DeepSeek) are essential to mitigate the precision loss and avoid gradient noise that can derail convergence.

Table 3. Trade-Offs of Different Precision in Pretraining Steps

Aspect	Memory	Communication	Compute Speed	Numerical Risk	Implementation
Conservative	Activations, weights	FP32	FP16	Low	Standard
Aggressive	Gradients, optimizer	FP8	FP8	Higher, requires robust scaling	Complex, requires custom kernels

Insight. Scaling laws reveal a relationship between model performance and scale (including parameters, data, and compute), which has served as a core theoretical foundation driving the development of LLMs. Guided by these laws, it has been possible to consistently improve performance by simply scaling up model size.

However, these conventional scaling laws face significant challenges in the domain of quantized training. A notable example is that low-bit models such as BitNet can be effectively trained at smaller scales (e.g., 3 billion parameters), but their performance fails to improve consistently as the model size increases, preventing successful scaling to larger parameter counts. This “scaling barrier” has become a critical issue limiting the training of quantized models.

To address this challenge, we could explore a novel training paradigm based on mixed-precision and dynamic fallback strategies to intelligently identify and preserve higher precision for critical modules or sensitive operations within a low-bit training framework. The most key problem is to overcome the current scaling limitations of low-bit models for scalable and efficient quantization training for large models.

5 Fine-Tuning with Quantization

Quantization fine-tuning refers to the process of further training or adjusting a model after pretraining, incorporating quantization techniques to optimize its performance at low precision^[58]. This primarily includes two paradigms: post-training quantization (PTQ) and quantization-aware training (QAT).

5.1 Post-Training Quantization

PTQ is a method that converts model parameters and/or activations to low-precision representations after the model has completed high-precision training^[59–61], requiring no additional training or only a small amount of calibration data. The prominent advantages of PTQ lie in its simplicity of implementation, low cost, and rapid deployment, making it particularly suitable for LLMs, which are extremely expensive to train and difficult to fully retrain. PTQ typically works by analyzing the statistical distribution of model parameters and/or activations offline, calculating appropriate quantization parameters (such as scaling factors and zero points), and then directly mapping floating-point values to low-precision integers.

Despite its efficiency, PTQ’s performance can significantly degrade at extremely low bit-widths^[5, 62, 63], especially when dealing with common activation outliers in LLMs. This is because outliers drastically stretch the quantization range, causing most non-outlier values to be assigned to very few quantization levels, thus losing precision. Furthermore, PTQ typically cannot change the numerical distribution formed during the model’s training process; therefore, its accuracy ceiling is limited by the characteristics of the original model.

5.2 Quantization-Aware Training

QAT simulates quantization effects during training or fine-tuning, allowing the model to adapt to low-precision weights. It typically achieves higher accuracy than PTQ.

Differentiable quantization is the core of QAT^[64]. QAT and learned step-size quantization (LSQ) bypass the gradient issue of rounding operations using the straight-through estimators (STEs) and jointly learn quantization parameters. By computing high-precision gradients and updating low-rank adaptors during backpropagation, it proves the effectiveness of model fine-tuning, which is typically achieved through the following steps.

1) *Insert Fake Quantization Nodes*^[65]. In the forward pass path of the model, fake quantization nodes are inserted after weights and activations. These nodes quantize floating-point values to low-precision integers and then dequantize them back to floating-point numbers. This way, the model “sees” quantized values during training, but all computations are still performed in the floating-point domain.

2) *Gradient Flow*. During backpropagation, fake quantization operations are usually treated as identity functions, or STEs are used to approximate gradients, allowing gradients to flow through quantization boundaries and update floating-point weights.

3) *Optimizer Update*. The optimizer updates the model’s floating-point weights based on the calculated floating-point gradients. At the end of training, these floating-point weights are finally quantized to low-precision integers.

To address the data dependency problem of QAT in LLMs, LLM-QAT^[66] proposes a data-free distillation method. Its core idea is the self-generated data to utilize the pre-trained high-precision LLM model itself to generate a small amount of synthetic data. This data can simulate real data distribution and be

used for QAT calibration and fine-tuning LLM-QAT, by integrating fake quantization operations into the fine-tuning process of the LLaMA model and combining data-free distillation, successfully quantizes weights and KV cache to 4-bit and activations to 6-bit, achieving significantly better performance than PTQ methods on various benchmarks. This demonstrates that QAT is an important way to achieve deep quantization for LLMs, especially in data-constrained scenarios, where data-free distillation strategies provide a feasible path for LLMs' QAT.

The most important aspect of QAT, in summary, is the quantized training with low precision gradient flow. One state-of-the-art work is BitNet^[24]. BitNet introduces an innovative paradigm where both weights and activations are quantized to extremely low bitwidths (e.g., 1 bit or 2 bits) during training, while still enabling effective learning through carefully designed gradient computation and update mechanisms.

Fig.9 illustrates the difference between QAT and PTQ. The core difference is the precision of backward gradient. In QAT, all parameters are quantized to low precision while training. This requires the computing of backward and updating parameters with low precision. In contrast, PTQ performs quantization only after the model has been fully trained with high precision. During the training phase of PTQ, all computations, including backward passes and parameter updates, are conducted using the original high precision (e.g., FP32). Quantization is applied as a post-processing step, converting the pre-trained high-precision weights and activations to lower precision (e.g., INT8) without further training adjustments.

This distinction leads to key trade-offs: QAT typi-

cally preserves higher accuracy by allowing the model to adapt to quantization effects during training, albeit with increased computational cost due to the need for retraining with quantized parameters. PTQ, on the other hand, is more computationally efficient as it avoids retraining, but may suffer greater accuracy degradation, especially for models sensitive to precision reduction, since it lacks the adaptive learning process under quantization constraints.

Insight. Achieving extremely low-bit quantization represents a paramount objective within the field of model compression. However, both PTQ and QAT exhibit a pronounced degradation in accuracy at such aggressive bit-widths during fine-tuning. To address this, it is imperative to design differentiable gradient flows that are effective under low-bit constraints to enable more accurate fine-tuning.

6 Inference with Quantization

Quantization at the inference stage is a critical step for deploying LLMs, aiming to maximize inference speed and minimize memory footprint while maintaining model performance. The most effective way to accelerate inference is weight-only quantization (WOQ).

6.1 8 or 4 Bits Quantization

Early WOQ work mostly adopted uniform symmetric quantization and used the same quantization parameters (scale) for the entire weight tensor (tensor-wise) or each layer (layer-wise). However, due to its inability to capture the non-uniformity of weight distribution, this coarse-grained approach leads to sig-

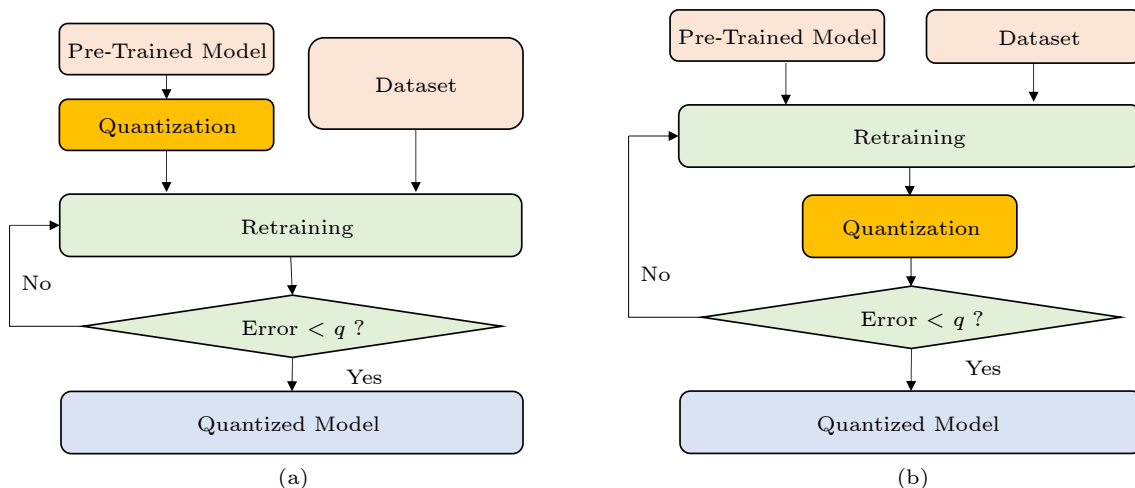


Fig.9. Comparison between (a) QAT and (b) PTQ.

nificant accuracy loss when dealing with extremely low bit-widths (e.g., below 4 bits).

To address this problem, group-wise quantization was proposed and has become the mainstream method. In GPTQ^[5], Frantar *et al.* gained insights through the Hessian matrix, enabling accurate INT3 or INT4 quantization for transformer models at the layer-wise and group-wise levels. This marked the first time high-precision PTQ-WOQ was achieved for large models. Similarly, Lin *et al.*^[6] found that not all weights are equally important. They proposed AWQ which identifies scaling factors to protect the salient channels in weights under the guidance of activations, realizing PTQ INT4 quantization without backpropagation. Its channel-wise scaling strategy is a fine-grained protection scheme.

Taking a step further, the NF4 (normal float 4) format introduced by Dettmers *et al.* in QLoRA^[7] is a non-uniform quantization scheme. This data type is specifically designed for weights with a normal distribution and combines double quantization to perform secondary compression on quantization constants, achieving breakthrough progress in 4-bit fine-tuning. The BitsAndBytes^[8] library implements algorithms such as LLM.int8()^[8] and NF4, promoting the practical application of WOQ.

Regarding weight-activation quantization, the key challenge lies in mitigating outliers during inference. Methods in this area have evolved from simple early-stage calibration techniques to more sophisticated solutions, such as outlier handling approaches (e.g., SmoothQuant^[14], QServe^[67]), advanced rounding strategies (e.g., AdaRound)^[68–70], and mature QAT frameworks^[71, 72]. Collectively, these efforts have driven the large-scale application of INT8/FP8 and even INT4/FP4 quantization in vision and natural language processing models.

Mixed-precision quantization offers another effective approach for addressing outliers. The main challenge of efficient mixed-precision computing comes from the dynamism and sparsity of outliers. First, the location of outliers is always changing during inference and cannot be determined ahead of time. The outlier varies with activation, which leads to its dynamical location during the inference and training and cannot be fully determined ahead of time. For example, BitsAndBytes^[8] is not efficient at runtime due to the computational overhead introduced by on-the-fly outlier detection, which leads to low hardware utilization.

Even in different blocks of a certain LLM, the distribution of outliers is even quite different. For instance, the distribution of outliers in the attention layer is quite different from that in the multilayer perceptron (MLP) layer. Dynamic location makes it necessary to identify outliers at runtime.

To address this problem, QUIK^[30], a reordered-based technique for efficient mixed-precision computing was proposed. This technique demonstrates impressive performance improvements, achieving up to 1.91x speedup over existing kernels of AutoAWQ^[6] on larger batches and up to 1.94x throughput gain on representative LLM models across various NVIDIA GPU devices.

Atom^[31] employs a special combination of fine-grained and mixed-precision quantization to preserve exceptional accuracy while dramatically reducing computational requirements. The key innovation in ATOM is its comprehensive approach to quantization, which includes multiple strategies to ensure optimal performance.

MixQ^[32] is a mixed-precision inference system that supports both 8-bit and 4-bit quantization. The system simultaneously quantizes both weights and activations, leveraging low-precision tensor cores (INT8/INT4 TensorCore) to accelerate inference while significantly improving system throughput. A key innovation in MixQ is its approach to handling outlier values in activation matrices. MixQ demonstrates performance gains in experimental evaluations. When testing inference on the LLaMA 70B model, MixQ achieved 1.9x better performance compared with traditional BitsAndBytes. Specifically, MixQ’s 4-bit kernel performance reached 724 TFLOPs, which is 3.13x higher than FP16, making the entire system excel when handling high-concurrency requests.

6.2 Aggressive Quantization

In the field of ultra-low bit-widths (e.g., 2 bits, 1 bit), binary/ternary neural networks represent an extreme form of WOQ. Early work like BinaryConnect^[9] and XNOR-Net^[10] demonstrated the feasibility of training 1-bit weight models. In recent years, BiLLM^[11] has for the first time applied 1-bit weight quantization to LLMs. Through well-designed double quantization scaling and residual error minimization, it maintains impressive language capabilities at a bit-width of 1.58 bits while the challenge is the maintenance of accuracy as shown in Fig.10.

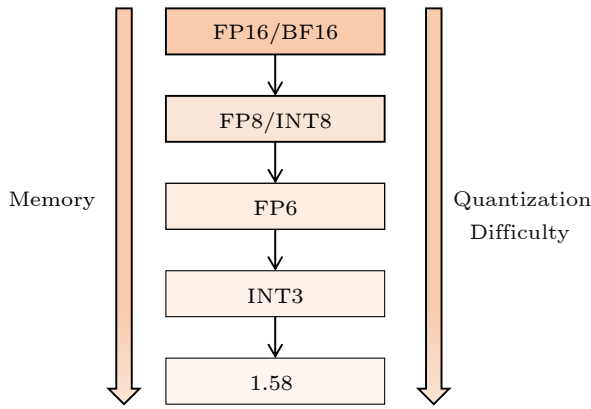


Fig.10. Different bits of quantization.

6.3 Hardware-Algorithm Co-Design

An optimal WOQ scheme must take into account the efficiency and feasibility of hardware deployment. Hardware-friendly algorithm design is crucial in this regard. Different layers exhibit varying sensitivities to quantization. Hardware-aware quantization^[62] automatically assigns different quantization bit-widths to different layers by analyzing the sensitivity of the Hessian matrix, maximizing compression and speedup while preserving accuracy.

Moreover, we need to take into account the compilation and kernel optimization. Deep learning compilers such as TensorRT^[37] and TVM^[38] provide WOQ kernels optimized for specific hardware (e.g., NVIDIA GPUs, ARM CPUs). These kernels support fast matrix multiplication for data types like INT8/INT4, serving as a key bridge to translate WOQ research into practical performance gains.

SqueezeLLM^[22] is a hardware-aware algorithm and a corresponding hardware architecture that co-designs a new quantization scheme and a specialized engine to compute softmax entirely in low-precision integer arithmetic. This breakthrough unlocks the full potential of quantization for LLMs by removing the softmax bottleneck, leading to much faster and more energy-efficient inference.

Insight. LLM inference quantization has been studied extensively, ranging from 8-bit down to very low bit-widths (such as 1.58-bit). FP8 currently offers the highest accuracy, and mixed precision is a highly effective method for maintaining accuracy at low bit-widths. However, all quantization kernels must be carefully designed to achieve an end-to-end speedup. The most challenging part is how to design quantization kernels with expert-level experience to optimize performance.

7 KV Cache Quantization

In the self-attention mechanism of LLMs, the Key (K) and Value (V) vectors^[73] corresponding to each generated token are computed and stored in the KV cache for use in the attention computation of subsequent tokens. By preserving the K and V vectors of all historical tokens, the KV cache avoids the need for the model to recompute historical attention when generating the current token, significantly improving inference efficiency. However, as the length of the generated sequence increases, the memory footprint of the KV cache grows linearly^[74], becoming a core memory bottleneck for the long-sequence inference in LLMs. Particularly in multi-user concurrent scenarios, this issue severely limits system throughput and the maximum supported context length. Therefore, KV cache quantization technology has become a key approach to enhance LLM inference efficiency and expand context window capabilities.

KV cache quantization faces three core challenges. 1) Large and Dynamic Range. The activation values (K and V vectors) in the KV cache typically have a large dynamic range, and their distribution changes dynamically with increasing sequence length and evolving generated content, making it difficult for static quantization ranges to adapt. 2) Outlier Problem. As shown in Fig.11, similar to regular activation values, outliers exist in the KV cache, and these outliers significantly affect quantization accuracy. 3) Precision Sensitivity. The precision of the KV cache is crucial for the generation quality and consistency of LLMs, and excessive quantization may lead to a decline in model output quality.

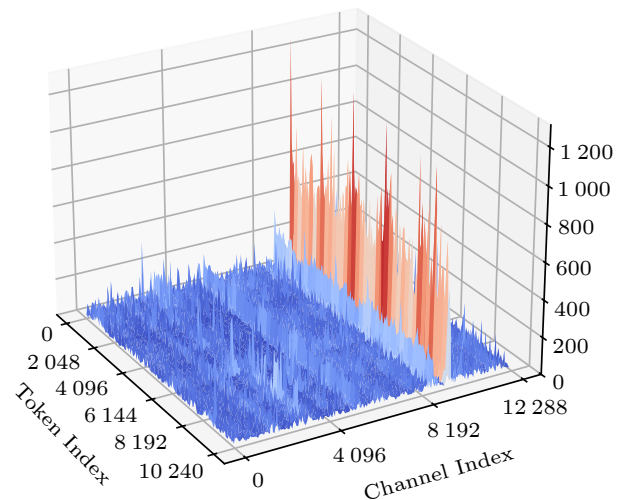


Fig.11. Distribution of outliers.

To address these challenges, researchers have proposed various KV cache quantization methods, as detailed below. Per-token quantization^[75–79] quantizes the KV cache token by token, and the K and V vectors of each token may have different statistical properties and can be quantized independently.

ZipCache^[80] achieves low-precision quantization through multiple innovative techniques, aiming to quantize the KV cache to the INT4 or INT8 precision while maintaining high accuracy. Its core technology adopts a non-uniform quantization strategy, i.e., assigning different numbers of quantization levels to different numerical ranges to more finely represent important values, especially suitable for scenarios with uneven numerical distribution. The Sage-Attention^[81] series represents a significant breakthrough in the plug-and-play attention quantization technology, which has been widely applied in open-source and commercial large models such as CogvideoX, Mochi, Flux, Llama3, and Qwen. SKVQ^[78] quantizes the KV cache using a sliding window mechanism, which is particularly suitable for handling long sequences and dynamically changing contexts. By performing local quantization within the sliding window, it can better adapt to the local statistical characteristics of the KV cache, while discarding old KV cache outside the window to further save memory. AQUA-KV is an adaptive KV cache quantization method that utilizes a compact linear predictor to mine internal and external correlations in the KV cache, thereby achieving more effective compression. It can adaptively adjust the quantization strategy according to the dynamic characteristics of the KV cache, balancing compression efficiency and precision.

8 Generating High Performance Quantization Kernels

Generating high-performance quantization kernels is an extremely challenging task, stemming from the complex interplay between the intricate storage patterns of low-precision data types, the specific constraints of hardware architectures, and the design of efficient computational pipelines. Although numerous research efforts have attempted to address these challenges from various angles, a general, efficient, and user-friendly kernel generation solution remains elusive.

8.1 Challenges in Kernel Generation

Quantization kernels must fully leverage the par-

allel architecture and memory hierarchy of the target hardware (e.g., GPUs). This involves fine-grained control over thread cooperation, data locality (e.g., caching data blocks via shared memory), and memory access patterns (avoiding bank conflicts). Simultaneously, kernels need to efficiently handle operations specific to quantized data, such as dequantization (converting low-precision weights to computation precision) and mixed-precision computation. Different quantization schemes (e.g., weight-only, weight-activation, non-uniform quantization) and bit-widths (e.g., 4-bit, 8-bit, or even non-power-of-two like 3-bit, 5-bit) further increase the complexity of kernel design, often requiring custom optimizations for each specific combination.

Existing approaches for high-performance quantization kernels can be broadly categorized as follows.

Hand-Optimized Kernels. For instance, the Marlin kernel^[82] is deeply optimized for Ampere and newer GPU architectures, achieving near-peak performance, particularly in batched inference scenarios. The kernel effectively resolves shared memory bank conflicts during dequantization through its quantization-aware interleaved layout and conflict-free shared memory access design. While such hand-tuned kernels deliver exceptional performance, they suffer from long development cycles and poor scalability, making them difficult to adapt to rapidly evolving quantization algorithms and hardware.

Programming Models or Compilers. To improve development efficiency, several programming models and compilers have been proposed^[83]. Triton^[84] provides high-level, Python-like abstractions, allowing developers to write GPU code at the tile level, while the compiler handles low-level optimizations. Tilus^[85], as a domain-specific language, further supports low-precision data types with arbitrary bit-widths from 1 to 8 bits. Its thread-block-level programming model and hierarchical memory space aim to simplify the development of efficient low-precision kernels.

Deep learning compilers like TVM attempt to compile pre-quantized models to various hardware backends by introducing quantization dialects and corresponding graph-level optimizations. Passes such as MergeSharedMemoryAllocations, which optimize shared memory reuse through liveness analysis, are critical for performance.

8.2 Automated Kernel Generation

Frameworks like Ladder^[83], Tilus^[85], and QFact-

or^[86] focus on automatically generating optimized low-precision kernels by decoupling the computational description from hardware-specific optimizations.

Recently, agent frameworks leveraging LLMs (as explored by frameworks like robust-kbench) have begun to investigate automated processes for CUDA kernel discovery, validation, and optimization, showing preliminary potential.

Despite these advancements, current methods still face issues of fragmentation. Most solutions are tailored to specific quantization formats (e.g., Marlin for 4-bit weights), hardware, or algorithms, lacking generality. Compiler abstractions can sometimes restrict critical low-level optimizations, while the optimization capabilities and reliability of automated frameworks in complex scenarios require further improvement.

Insight. Generating high-performance quantization kernels is very challenging. For the memory-bound part, the entire pipeline, including dequantization, quantization, and computation must be carefully organized to maximize bandwidth utilization. In the compute-bound part, low-precision computing units should overlap with other operations. However, most existing works fail to provide such fine-grained optimizations. Combining auto-generation techniques with human expertise is essential to designing more efficient kernels.

9 Conclusions

This paper comprehensively surveyed the latest advancements in large language model (LLM) quantization, covering various stages from pretraining to inference. It delved into FP8 and FP4 quantization during pretraining, post-training quantization (PTQ), and quantization-aware training (QAT) in quantization fine-tuning, and various quantization methods during inference, including weight-only, weight-activation, activation quantization with outliers, and KV cache quantization. Besides, it analyzed the principles, advantages, and challenges of each method and their applications in LLMs.

The development of quantization techniques has significantly propelled the deployment and application of these massive models in resource-constrained environments, making them accessible to a wider range of users and scenarios. From low-precision floating-point formats in the pretraining phase, to PTQ and QAT in the fine-tuning phase, and to fine-grained quantization for weights, activations, and KV cache during inference, researchers are continuously explor-

ing how to maximize compression and acceleration while preserving model performance. Particularly in addressing the prevalent outlier problem in LLM activations.

Despite significant achievements, LLM quantization still faces numerous challenges, such as accuracy loss in low-bit quantization, the generality of outlier handling, and hardware support and software stack optimization. Future research will focus on developing more refined and adaptive quantization strategies, strengthening hardware-software co-design, advancing differentiable quantization and automated quantization, and exploring the effective combination of quantization with other model compression techniques. As these challenges are gradually overcome, we have reason to believe that quantization technology will continue to play a crucial role in promoting the widespread adoption and application of LLMs, making the powerful capabilities of artificial intelligence more accessible.

Conflict of Interest Ji-Dong Zhai is an editorial board member for the Journal of Computer Science and Technology and was not involved in the editorial review of this article. The authors declare that there are no other competing interests.

References

- [1] Brown T B, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler D M, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodio D. Language models are few-shot learners. In *Proc. the 34th International Conference on Neural Information Processing Systems*, Dec. 2020, Article No. 159. DOI: [10.5555/3495724.3495883](https://doi.org/10.5555/3495724.3495883).
- [2] Touvron H. LLaMA: Open and efficient foundation language models. arXiv:2110.02861, 2023. <https://arxiv.org/abs/2110.02861>, Jan. 2026.
- [3] BigScience Workshop. BLOOM: A 176B-parameter open-access multilingual language model. arXiv:2211.05100, 2022. <https://arxiv.org/abs/2211.05100>, Jan. 2026.
- [4] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024. <https://arxiv.org/pdf/2412.19437>, Jan. 2026.
- [5] Frantar E, Ashkboos S, Hoefler T, Alistarh D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv:2210.17323, 2022. <https://arxiv.org/abs/2210.17323>, Jan. 2026.
- [6] Lin J, Tang J, Tang H, Yang S, Dang X, Han S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *GetMobile: Mobile Com-*

- puting and Communications, 2024, 28(4): 12-17. DOI: [10.1145/3714983.3714987](https://doi.org/10.1145/3714983.3714987).
- [7] Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L. QLoRA: Efficient finetuning of quantized LLMs. In *Proc. the 37th International Conference on Neural Information Processing Systems*, Dec. 2023, pp.10088–10115. DOI: [10.5555/3666122.3666563](https://doi.org/10.5555/3666122.3666563).
 - [8] Dettmers T, Lewis M, Belkada Y, Zettlemoyer L. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Proc. the 36th International Conference on Neural Information Processing Systems*, Nov. 28–Dec. 9, 2022, pp.30318–30332. DOI: [10.5555/3600270.3602468](https://doi.org/10.5555/3600270.3602468).
 - [9] Courbariaux M, Bengio Y, David J P. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Proc. the 29th International Conference on Neural Information Processing Systems*, Dec. 2015, pp.3123–3131. DOI: [10.5555/2969442.2969588](https://doi.org/10.5555/2969442.2969588).
 - [10] Rastegari M, Ordonez V, Redmon J, Farhadi A. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Proc. the 14th European Conference on Computer Vision*, Oct. 2016, pp.525–542. DOI: [10.1007/978-3-319-46493-0_32](https://doi.org/10.1007/978-3-319-46493-0_32).
 - [11] Huang W, Liu Y, Qin H, Li y, Zhang S, Liu X, Magno M, Qi X. BiLLM: Pushing the limit of post-training quantization for LLMs. arXiv:2402.04291, 2024. <https://arxiv.org/abs/2402.04291>, Jan. 2026.
 - [12] Nagel M, Amjad R A, Van Baalen M, Louizos C, Blankevoort T. Up or down? Adaptive rounding for post-training quantization. In *Proc. the 37th International Conference on Machine Learning*, Jul. 2020, pp.7197–7206. DOI: [10.5555/3524938.3525605](https://doi.org/10.5555/3524938.3525605).
 - [13] Li Y, Gong R, Tan X, Yang Y, Hu P, Zhang Q, Yu F, Wang W, Gu S. BRECQ: Pushing the limit of post-training quantization by block reconstruction. arXiv:2102.05426, 2021. <https://arxiv.org/abs/2102.05426>, Jan. 2026.
 - [14] Xiao G, Lin J, Seznec M, Wu H, Demouth J, Han S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proc. the 40th International Conference on Machine Learning*, Jul. 2023, pp.38087–38099. DOI: [10.5555/3618408.3619993](https://doi.org/10.5555/3618408.3619993).
 - [15] Frantar E, Alistarh D. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proc. the 40th International Conference on Machine Learning*, Jul. 2023, pp.10323–10337. DOI: [10.5555/3618408.3618822](https://doi.org/10.5555/3618408.3618822).
 - [16] Gregory K, Miller A. C++ AMP: Accelerated massive parallelism with Microsoft Visual C++. Technical Report, Microsoft Press Corp., 2012. <https://learn.microsoft.com/zh-cn/cpp/parallel/amp>, Jan. 2026.
 - [17] NVIDIA. Transformer Engine. Technical Report, NVIDIA Corporation, 2025. <https://docs.nvidia.com/deeplearning/transformer-engine>, Jan. 2026.
 - [18] Lee C, Jin J, Cho Y, Park E. QEFT: Quantization for efficient fine-tuning of LLMs. In *Proc. the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Nov. 2024, pp. 13823–13837. DOI: [10.18653/v1/2024.findings-emnlp.811](https://doi.org/10.18653/v1/2024.findings-emnlp.811).
 - [19] FAIR. Torchao. Technical Report, Pytorch Team, 2026. <https://docs.pytorch.org/ao/stable/index.html>, Jan. 2026.
 - [20] Li Y, Yu Y, Liang C, He P, Karampatziakis n, Chen W, Zhao T. LoftQ: LoRA-fine-tuning-aware quantization for large language models. arXiv:2310.08659, 2023. <https://arxiv.org/abs/2310.08659>, Jan. 2026.
 - [21] Xia H, Zheng Z, Wu X et al. FP6-LLM: Efficiently serving large language models through FP6-centric algorithm-system co-design. arXiv:2401.14112, 2024. <https://arxiv.org/abs/2401.14112>, Jan. 2026.
 - [22] Kim S, Hooper C, Gholami A, Dong Z, Li X, Shen S, Mahoney MW, Keutzer K. SqueezeLLM: Dense-and-sparse quantization. arXiv:2306.07629, 2023. <https://arxiv.org/abs/2306.07629>, Jan. 2026.
 - [23] Lee C, Jin J, Kim T, Kim H, Park E. OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(12): 13355–13364. DOI: [10.1609/aaai.v38i12.29237](https://doi.org/10.1609/aaai.v38i12.29237).
 - [24] Wang H, Ma S, Dong L, Huang S, Wang H, Ma L, Yang F, Wang R, Wu Y, Wei F. BitNet: Scaling 1-bit transformers for large language models. arXiv: 2310.11453, 2023. <https://arxiv.org/abs/2310.11453>, Jan. 2026.
 - [25] Peng H, Wu K, Wei Y, Zhao G, Yang Y, Liu Z, Xiong Y, Yang Z, Ni B, Hu J, Li R, Zhang M, Li C, Ning J, Wang R, Zhang Z, Liu S, Chau J, Hu H, Cheng P. FP8-LM: Training FP8 large language models. arXiv: 2310.18313, 2023. <https://arxiv.org/abs/2310.18313>, Jan. 2026.
 - [26] Liu Z, Zhao C, Fedorov I. SpinQuant: LLM quantization with learned rotations. arXiv:2405.16406, 2024. <https://arxiv.org/abs/2405.16406>, Jan. 2026.
 - [27] Yuan Z, Niu L, Liu J et al. RPTQ: Reorder-based post-training quantization for large language models. arXiv:2304.01089, 2023. <https://arxiv.org/abs/2304.01089>, Jan. 2026.
 - [28] Zhang Y, Zhang P, Huang M et al. QQQ: Quality quantization for large language models. arXiv:2406.09904, 2024. <https://arxiv.org/abs/2406.09904>, Jan. 2026.
 - [29] Shao W, Chen M, Zhang Z, Xu P, Zhao L, Li Z, Zhang K, Gao P, Qiao Y, Luo P. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *Proc. the 12th International Conference on Learning Representations*, May 2024.
 - [30] Ashkboos S, Markov I, Frantar E, Zhong T, Wang X, Ren J, Hoefler T, Alistarh D. QUIK: Towards end-to-end 4-bit inference on generative large language models. In *Proc. the 2024 Conference on Empirical Methods in Natural Language Processing*, Nov. 2024, pp.3355–3371. DOI: [10.18653/v1/2024.emnlp-main.197](https://doi.org/10.18653/v1/2024.emnlp-main.197).
 - [31] Zhao Y, Lin C Y, Zhu K, Ye Z, Chen L, Zheng S, Ceze L, Krishnamurthy A, Chen T, Kasikci B. Atom: Low-bit quantization for efficient and accurate LLM serving. In *Proc. the 7th Annual Conference on Machine Learning and Systems*, May 2024.
 - [32] Chen Y, Zhang C, Dong R, Zhang H, Zhang Y, Lu Z, Zhai J. MixQ: Taming dynamic outliers in mixed-precision quantization by online prediction. In *Proc. the 2024*

- SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, Nov. 2024, pp.1–15. DOI: [10.1109/SC41406.2024.00080](https://doi.org/10.1109/SC41406.2024.00080).
- [33] Jacob B, Kligys S, Chen B, Zhu M, Tang M, Howard A, Adam H, Kalenichenko D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proc. the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Jun. 2018, pp.2704–2713. DOI: [10.1109/CVPR.2018.00286](https://doi.org/10.1109/CVPR.2018.00286).
- [34] Esser S K, McKinstry J L, Bablani D, Appuswamy R, Modha D S. Learned step size quantization. In *Proc. the 8th International Conference on Learning Representations*, Apr. 2020.
- [35] Dong Z, Yao Z, Gholami A, Mahoney M W, Keutzer K. HAWQ: Hessian AWARE quantization of neural networks with mixed-precision. In *Proc. the 2019 IEEE/CVF International Conference on Computer Vision*, Oct. 27–Nov. 2, 2019, pp.293–302. DOI: [10.1109/ICCV.2019.00038](https://doi.org/10.1109/ICCV.2019.00038).
- [36] Dettmers T, Lewis M, Shleifer S, Zettlemoyer L. 8-bit optimizers via block-wise quantization. arXiv: 2110.02861, 2022. <https://arxiv.org/abs/2110.02861>, Jan. 2026.
- [37] NVIDIA. NVIDIA TensorRT. NVIDIA Corporation, 2023. <https://developer.nvidia.com/tensorrt>, Mar. 2024.
- [38] Chen T, Moreau T, Jiang Z, Zheng L, Yan E Q, Shen H, Cowan M, Wang L, Hu Y, Ceze L, Guestrin C, Krishnamurthy A. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. the 13th USENIX Conference on Operating Systems Design and Implementation*, Oct. 2018, pp.578–594. DOI: [10.5555/3291168.3291211](https://doi.org/10.5555/3291168.3291211).
- [39] Wu Z, Chen J, Zhong H, Huang D, Wang Y. AdaLog: Post-training quantization for vision transformers with adaptive logarithm quantizer. In *Proc. the 8th European Conference on Computer Vision*, Sept. 29-Oct. 4, 2025, pp.411–427. DOI: [10.1007/978-3-031-73383-3_24](https://doi.org/10.1007/978-3-031-73383-3_24).
- [40] Zheng X, Qin H, Li Y, Chu H, Wang J, Guo J, Magno M, Liu X. First-order error matters: Accurate compensation for quantized large language models. arXiv: 2507.11017, 2025. <https://arxiv.org/abs/2507.11017>, Jan. 2026.
- [41] Huang W, Qin H, Liu Y, Li Y, Liu Q, Liu X, Benini L, Magno M, Zhang S, Qi X. Slim-LLM: Saliency-driven mixed-precision quantization for large language models. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [42] Moon J, Kim D, Cheon J, Ham B. Instance-aware group quantization for vision transformers. In *Proc. the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2024, pp. 16132–16141. DOI: [10.1109/CVPR52733.2024.01527](https://doi.org/10.1109/CVPR52733.2024.01527).
- [43] Qin H, Ma X, Zheng X, Li X, Zhang Y, Liu S, Luo J, Liu X, Magno M. Accurate LoRA-Finetuning quantization of LLMs via information retention. In *Proc. the 41st International Conference on Machine Learning*, Jul. 2024, pp.41498–41516. DOI: [10.5555/3692070.3693757](https://doi.org/10.5555/3692070.3693757).
- [44] Fang J, Shafiee A, Abdel-Aziz H, Thorsley D, Georgiadis G, Hassoun J. Post-training quantization for deep neural networks. arXiv:2002.00104, 2020. <https://arxiv.org/abs/2002.00104>, Jan. 2026.
- [45] Yang J, Li Z, Feng Z, Xie Y. A survey on neural network quantization. In *Proc. the 6th International Conference on Computer Information and Big Data Applications*, Mar. 2025, pp.384–394. DOI: [10.1145/3746709.3746773](https://doi.org/10.1145/3746709.3746773).
- [46] Wu D, Tang Q, Zhao Y, Zhang M, Fu Y, Zhang D. EasyQuant: Post-training quantization via scale optimization. arXiv: 2006.16669, 2020. <https://arxiv.org/abs/2006.16669>, Jan. 2026.
- [47] Lv C, Chen H, Guo J, Guo J, Guo J, Ding Y, Liu X, Liu X, Liu X. PTQ4SAM: Post-training quantization for segment anything. In *Proc. the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Jun. 2024, pp.15941–15951. DOI: [10.1109/CVPR52733.2024.01509](https://doi.org/10.1109/CVPR52733.2024.01509).
- [48] Shang Y, Yuan Z, Xie B, Wu B, Yan Y. Post-training quantization on diffusion models. In *Proc. the 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Jun. 2023, pp.1972–1981. DOI: [10.1109/CVPR52729.2023.00196](https://doi.org/10.1109/CVPR52729.2023.00196).
- [49] Wang W, Shubham R, De La Parra C, Kumar A. MixA-Q: Revisiting activation sparsity for vision transformers from a mixed-precision quantization perspective. In *Proc. the 2025 IEEE/CVF International Conference on Computer Vision*, Oct. 2024, pp. 22143–22152.
- [50] Badri H, Shaji A. Half-Quadratic Quantization of large machine learning models. Technical Report, Mobius Labs GmbH. 2025. https://dropbox.github.io/hqq_blog/, Jan. 2026.
- [51] Egiazarian V, Panferov A, Kuznedev D, Frantar E, Babenko A, Alistarh D. Extreme compression of large language models via additive quantization. In *Proc. the 41st International Conference on Machine Learning*, Jul. 2024, Article No. 488. DOI: [10.5555/3692070.3692558](https://doi.org/10.5555/3692070.3692558).
- [52] Li S, Ning X, Wang L, Liu T, Shi X, Yan S, Dai G, Yang H, Wang Y. Evaluating quantized large language models. In *Proc. the 41st International Conference on Machine Learning*, Jul. 2024, Article No. 1144. DOI: [10.5555/3692070.3693214](https://doi.org/10.5555/3692070.3693214).
- [53] Li Y, Yin R, Lee D, Xiao S, Panda P. GPTAQ: Efficient finetuning-free quantization for asymmetric calibration. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [54] Wang R, Gong Y, Liu X, Zhao G, Yang Z, Guo B, Zha Z J, Cheng P. Optimizing large language model training using FP4 quantization. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [55] Mishra A, Stosic D, Layton S, Micikevicius P. Recipes for pre-training LLMs with MXFP8. arXiv: 2506.08027, 2025. <https://arxiv.org/abs/2506.08027>, Jan. 2026.
- [56] Xi H, Cai H, Zhu L, Lu Y, Keutzer K, Chen J, Han S. COAT: Compressing optimizer states and activations for memory-efficient FP8 training. In *Proc. the 13th International Conference on Learning Representations*, Apr. 2025.
- [57] Zhang Y, Zhen H L, Yuan M, Yu B. MOSS: Efficient and accurate FP8 LLM training with microscaling and automatic scaling. arXiv: 2511.05811, 2025. <https://arxiv.org/abs/2511.05811>, Jan. 2026.

- [abs/2511.05811](#), Jan. 2026.
- [58] Kim J, El Halabi M, Park W, Schaefer C J S, Lee D, Park Y, Lee J W, Song H O. GuidedQuant: Large language model quantization via exploiting end loss guidance. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [59] Liu T, Yuan J, Jin H, Zhong S, Xu Z, Braverman V, Chen B, Hu X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proc. the 41st International Conference on Machine Learning*, Jul. 2024, pp.32332–32344.
- [60] Tao W, Lu H, Qu X, Zhang B, Lu K, Wan J, Wang J. MoQAE: Mixed-precision quantization for long-context LLM inference via mixture of quantization-aware experts. In *Proc. the 63rd Annual Meeting of the Association for Computational Linguistics*, Jul. 2025, pp.10810–10820. DOI: [10.18653/v1/2025.acl-long.531](#).
- [61] Saxena U, Sharify S, Roy K, Wang X. ResQ: Mixed-precision quantization of large language models with low-rank residuals. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [62] Zhang Y, Wang M, Zou L, Liu W, Zhen H L, Yuan M, Yu B. MixPE: Quantization and hardware co-design for efficient LLM inference. arXiv: 2411.16158, 2025. [https://arxiv.org/abs/2411.16158](#), Jan. 2026.
- [63] Yao Z, Aminabadi R Y, Zhang M, Wu X, Li C, He Y. ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. In *Proc. the 36th International Conference on Neural Information Processing Systems*, Nov. 28–Dec. 9, 2022, Article No. 1970. DOI: [10.5555/3600270.3602240](#).
- [64] Chen M, Shao W, Xu P, Wang J, Gao P, Zhang K, Luo P. EfficientQAT: Efficient quantization-aware training for large language models. In *Proc. the 63rd Annual Meeting of the Association for Computational Linguistics*, Jul. 27–Aug. 1, 2025, pp.10081–10100.
- [65] Pham C, Hoang A D, Nguyen C C, Le T, Phung D, Carneiro G, Do T T. MetaAug: Meta-data augmentation for post-training quantization. In *Proc. the 18th European Conference on Computer Vision*, Sept. 29–Oct. 4, 2024, pp.236–252. DOI: [10.1007/978-3-031-73383-3_14](#).
- [66] Liu Z, Oguz B, Zhao C et al. LLM-QAT: Data-free quantization aware training for large language models. In *Proc. the 62nd Annual Meeting of the Association for Computational Linguistics*, Aug. 2024, pp. 467–484 DOI: [10.18653/v1/2024.findings-acl.26](#).
- [67] Lin Y, Tang H, Yang S, Zhang Z, Xiao G, Gan C, Han S. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. arXiv:2405.04532, 2024. [https://arxiv.org/abs/2405.04532](#), Jan. 2026.
- [68] Dong S, Cheng W, Qin J, Wang W. QAQ: Quality adaptive quantization for LLM KV cache. arXiv:2403.04643, 2024. [https://arxiv.org/abs/2403.04643](#), Jan. 2026.
- [69] Xu X, Li C, Chen Y, Chang X, Liu J, Wang S. No token left behind: Efficient vision transformer via dynamic token idling. arXiv:2310.05654, 2023. [https://arxiv.org/abs/2310.05654](#), Jan. 2026.
- [70] Chee J, Cai Y, Kuleshov V, De Sa C. QuIP: 2-bit quantization of large language models with guarantees. arXiv:2307.13304, 2023. [https://arxiv.org/abs/2307.13304](#), Jan. 2026.
- [71] Thangarasa V, Gupta A, Marshall W, Li T, Leong K, De-Coste D, Lie S, Saxena S. SPDF: Sparse pre-training and dense fine-tuning for large language models. arXiv:2303.10464, 2023. [https://arxiv.org/abs/2303.10464](#), Jan. 2026.
- [72] Xu C, Yue y, Xu Z, Hu X, Yu J, Chen Z, Zhou S, Yuan Z, Yang D. RWKVQuant: Quantizing the RWKV family with proxy guided hybrid of scalar and vector quantization. arXiv:2505.03803, 2025. [https://arxiv.org/abs/arXiv:2505.03803](#), Jan. 2026.
- [73] Chen Z, Hu X, Yang D, Xu Z, Xu C, Yuan Z, Zhou S, Yu J. MoEQuant: Enhancing quantization for mixture-of-experts large language models via expert-balanced sampling and affinity guidance. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [74] Hooper C, Kim S, Mohammadzadeh H et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Proc. the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, Dec. 2024, pp. 1270–1303. DOI: [10.52202/079017-0040](#).
- [75] Zhang T, Yi J, Xu Z, Shrivastava A. KV cache is 1 bit per channel: Efficient large language model inference with coupled quantization. In *Proc. the 38th International Conference on Neural Information Processing Systems*, Dec. 2024, pp.3304–3331. DOI: [10.5555/3737916.3738025](#).
- [76] Zhang J, Zhu D, Song Y, Wu W, Kuang C, Li X, Shang L, Liu Q, Li S. More tokens, lower precision: Towards the optimal token-precision trade-off in KV cache compression. In *Proc. the 2025 Association for Computational Linguistics*, Nov. 2025, pp.8092–8105. DOI: [10.18653/v1/2025.findings-emnlp.429](#).
- [77] Shi L, Zhang H, Yao Y, Li Z, Zhao H. Keep the cost down: A review on methods to optimize LLM’s KV cache consumption. arXiv: 2407.18003, 2024. [https://arxiv.org/abs/2407.18003](#), Jan. 2026.
- [78] Duanmu H, Yuan Z, Li X, Duan J, Zhang X, Lin D. SKVQ: Sliding-window key and value cache quantization for large language models. arXiv: 2405.06219, 2024. [https://arxiv.org/abs/2405.06219](#), Jan. 2026.
- [79] Shutova A, Malinovskii V, Egiazarian V, Kuznedelev D, Mazur D, Surkov N, Ermakov I, Alistarh D. Cache me if you must: Adaptive key-value quantization for large language models. In *Proc. the 42nd International Conference on Machine Learning*, Jul. 2025.
- [80] He Y, Zhang L, Wu W, Liu J, Zhou H, Zhuang B. Zip-Cache: Accurate and efficient KV cache quantization with salient token identification. In *Proc. the 38th International Conference on Neural Information Processing Systems*, Dec. 2024, Article No. 2181.
- [81] Zhang J, Wei J, Zhang P, Zhu J, Chen J. SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration. In *Proc. the 13th International Conference on Learning Representations*, Apr. 2025.
- [82] Frantar E, Castro R L, Chen J, Hoefler T, Alistarh D.

MARLIN: Mixed-precision auto-regressive parallel inference on large language models. In *Proc. the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, Mar. 2025, pp.239–251.

- [83] Wang L, Ma L, Cao S, Zhang Q, Xue J, Shi Y, Zheng N, Miao Z, Yang F, Cao T, Yang Y, Yang M. Ladder: Enabling efficient low-precision deep learning computing through hardware-aware tensor transformation. In *Proc. the 18th USENIX Symposium on Operating Systems Design and Implementation*, Jul. 2024, pp.307–323. DOI: [10.5555/3691938.3691955](https://doi.org/10.5555/3691938.3691955).
- [84] Wood B, Azim A. Triton: A domain specific language for cyber-physical systems. In *Proc. the 22nd IEEE International Conference on Industrial Technology*, Mar. 2021, pp.810–816. DOI: [10.1109/ICIT46573.2021.9453575](https://doi.org/10.1109/ICIT46573.2021.9453575).
- [85] Ding Y, Hou B, Zhang X, Lin A, Chen T, Hao C Y, Wang Y, Pekhimenko G. Tilus: A tile-level GPGPU programming language for low-precision computation. arXiv: 2504.12984, 2025. <https://arxiv.org/abs/2504.12984>, Jan. 2026.
- [86] Zhang Q, Zhai M, Sun R, Zhai J. QFactory: Accelerating quantized large language model serving with Qtile graphs. In *Proc. the 2025 USENIX Conference on Usenix Annual Technical Conference*, Jul. 2025, Article No. 38. DOI: [10.5555/3768039.3768077](https://doi.org/10.5555/3768039.3768077).



Yi-Dong Chen received his Ph.D. degree from the Computer Network Information Center, University of Chinese Academy of Sciences, Beijing, in 2023. He is now a postdoctoral researcher in the Department of Computer Science and Technology, Tsinghua University, Beijing. He is now studying AI systems with heterogeneous architecture, including operator optimization, low-precision optimization and compiler optimization of LLM.



Kai-Jun Zheng conducted in-depth research on large model quantization and high-performance computing. He is researching and designing efficient mixed-precision data structures to enhance memory access efficiency. Specifically, He is developing a hardware-friendly mixed-precision data structure to optimize memory efficiency during the dequantization phase.



Zhen-Hua Guo received his Ph.D. degree from Harbin Engineering University, Harbin. He is currently a researcher with IEIT (Inspur Electronic Information Industry Co., Ltd.), Beijing. His research interests mainly include computer system architecture and artificial intelligence.



Qi-Hao Zhang received his B.S. degree in computer science and technology from Tsinghua University, Beijing, in 2023. He is currently a Ph.D. candidate in the PACMAN (Parallel Architecture and Compiler Technology of Mobile, Accelerated, and Networked Systems) Lab at Tsinghua University, Beijing. His research interests include machine learning systems and high performance computing, focusing on vertical optimization and hardware-software codesign for large-scale AI model training and inference.



Yong-Hua Zhang received his Ph.D. degree in computer science and technology at Beihang University, Beijing, in 2023. He joined the High-Performance Computing Institute at the Department of Computer Science and Engineering, Tsinghua University, Beijing, as a postdoctoral researcher in 2023. His research interests include AI model compression and compiler optimization, particularly for convolutional neural network and large language models (LLMs) on GPUs.



Ji-Dong Zhai received his B.S. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, in 2003, and his Ph.D. degree in computer science from Tsinghua University, Beijing, in 2010. He is a full professor with the Department of Computer Science and Technology, Tsinghua University, Beijing. His research interests include performance evaluation for high performance computers, performance analysis and modeling of parallel applications.